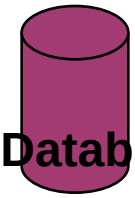
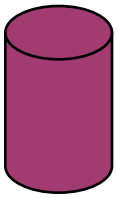
DATABASE STUDY GUIDE

Architecture, Models, and Systems





1. Database Fundamentals

A Database Management System (DBMS) is software that interacts with end-users, applications, and the database itself to capture and analyze data.

Key Historical Developments:
• Navigational DBMS (1960s): Hierarchical (Tree) and Network models
• Relational DBMS (1970s): Codd's 12 Rules, Table-based structures
• NoSQL (2000s): Designed for distributed data stores and scaling

Core Components:
- Data: Raw facts and figures
- Schema: The logical structure/blueprint of the database
- Engine: The underlying software component processing queries

Hierarchy of Data:

`Database > Table > Row (Tuple) > Column (Attribute)`

`Field > Byte > Bit`

Popular DBMS Examples:

System	Type	Language	Primary Use
PostgreSQL	Relational	SQL	Enterprise App
MySQL	Relational	SQL	Web Apps (LAMP)
MongoDB	NoSQL (Doc)	BSON/JS	Big Data/Flexibility
Redis	NoSQL (KV)	CLI	Caching/Session
Neo4j	Graph	Cypher	Social Networks

RELATIONAL MODEL & SQL

2. Relational Concepts (RDBMS)

The Relational Model organizes data into tables (relations) which can be linked based on data common to each.

1. KEYS (Constraints):
- Primary Key (PK): Unique identifier for a record (e.g., ID)
- Foreign Key (FK): Links to PK in another table (Relationship)
- Candidate Key: Any column that COULD be a Primary Key
- Composite Key: Two+ columns combined to create uniqueness

2. RELATIONSHIPS:
- One-to-One (1:1): User <-> Passport
- One-to-Many (1:N): User <-> Orders
- Many-to-Many (N:M): Students <-> Courses (Requires Join Table)

3. SQL Syntax

Basic Query Structure:

```
SELECT column1, column2
FROM table_name
WHERE condition = true
ORDER BY column1 ASC;
```

Common Commands (CRUD):

1. CREATE (Insert):
```
INSERT INTO Users (Name)
VALUES ('Alice');
```

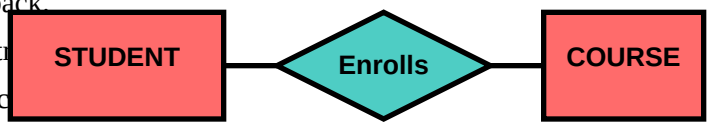
2. READ (Select):
```
SELECT * FROM Users;
```

3. UPDATE:
```
UPDATE Users SET age=25
WHERE id=1;
```

4. DELETE:
```
DELETE FROM Users WHERE id=1;
```

ACID Properties (Transactions):

- Atomicity: All or nothing. Complete success or full rollback.
- Consistency: Data remains valid according to rules/constr
- Isolation: Concurrent transactions don't interfere with eac
- Durability: Committed data is saved permanently, even after crash.



4. Normalization (Design Optimization)

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

Standard Normal Forms:
1. First Normal Form (1NF):
  • Atomic values (no lists in cells)
  • Unique column names

2. Second Normal Form (2NF):
  • Must be in 1NF
  • No partial dependencies (Attributes depend on WHOLE PK)

3. Third Normal Form (3NF):
  • Must be in 2NF
  • No transitive dependencies (A->B->C)
  • 'The Key, the whole Key, and nothing but the Key'

5. NoSQL Types:

Type	Data Model	Example	Best For
Document	JSON/BSON	MongoDB	Content Mgmt, Ca
Key-Value	Hash Map	Redis	Caching, Sessions
Columnar	Columns	Cassandra	Analytics, Time Se
Graph	Nodes/Edges	Neo4j	Social, Fraud Dete

6. CAP Theorem (Distributed Systems)

In a distributed data store, you can only guarantee 2 of 3:

• Consistency (C): Every read receives the most recent write or an error.

• Availability (A): Every request receives a (non-error) response.

• Partition Tolerance (P): System operates despite network failures.

Trade-offs:

- CA: RDBMS (MySQL) - Hard to scale horizontally

- CP: MongoDB/HBase - Data is consistent, might timeout

- AP: Cassandra/Dynamo - Always up, data might be 'eventually' consistent

PERFORMANCE & ADMINISTRATION

7. Indexing & Performance

Indexing is a data structure technique to quickly locate and access data in a database (like a book index).

Key Concepts:
• B-Tree: Balanced tree structure (Standard for most DBs)
• Hash Index: O(1) lookup, good for equality checks (=)
• Clustered Index: Sorts the actual data rows (Only 1 per table)
• Non-Clustered: Points to the data rows (Multiple allowed)

Query Optimization Tips:
1. Avoid SELECT * (Select specific columns)
2. Use Indexes on columns used in WHERE, JOIN, ORDER BY
3. Analyze Query Execution Plans (EXPLAIN)
4. Avoid Wildcards at start of string (LIKE '%text')

8. Scaling Strategies

```
Vertical Scaling (Scale Up):
• Adding CPU, RAM to existing server
• Limit: Hardware constraints
• Good for: Relational DBs

Horizontal Scaling (Scale Out):
• Adding more servers
• Requires Sharding/Partitioning
• Good for: NoSQL / Distributed SQL

Replication:
• Master-Slave: Writes to Master, Reads from
• Master-Master: Writes to any node
```

9. Logic in Database

Stored Procedures:
Precompiled SQL code stored on the server.
Benefits: Reduced network traffic, security.

Triggers:
Code that executes automatically in response to events (INSERT, UPDATE, DELETE).

Views:
Virtual tables based on the result-set of a query.
Simplifies complex joins for users.

10. Backup & Recovery

Full Backup: Complete copy of data.
Differential: Changes since last Full.
Incremental: Changes since last Backup.

Point-in-Time Recovery:
Restoring DB to a specific timestamp using Transaction Logs (Write-Ahead Logs).

ETL (Extract, Transform, Load):
Moving data from DB to Data Warehouse.

Key Takeaways:

1. RDBMS (SQL) is best for structured, transactional data (ACID)
2. NoSQL is best for unstructured, high-scale, distributed data
3. Normalization prevents data anomalies but adds complexity
4. Indexing is crucial for read-heavy performance
5. Design schemas based on query patterns, not just storage